

▼ Heart Disease Prediction

Importing the Dependencies

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Data Collection and Processing

```
# Loading the csv data to a Pandas DataFrame
heart_data = pd.read_csv('heart.csv')
```

```
# Print first 5 rows of the dataframe
print("First 5 rows:")
print(heart_data.head())
```

```
First 5 rows:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52   1   0    125    212   0         1     168       0       1.0       2
1   53   1   0    140    203   1         0     155       1       3.1       0
2   70   1   0    145    174   0         1     125       1       2.6       0
3   61   1   0    148    203   0         1     161       0       0.0       2
4   62   0   0    138    294   1         1     106       0       1.9       1

   ca  thal  target
0   2     3        0
1   0     3        0
2   0     3        0
3   1     3        0
4   3     2        0
```

```
# Print last 5 rows of the dataframe
print("\nLast 5 rows:")
print(heart_data.tail())
```

```
Last 5 rows:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  \
1020  59   1   1    140    221   0         1     164       1       0.0
1021  60   1   0    125    258   0         0     141       1       2.8
1022  47   1   0    110    275   0         0     118       1       1.0
1023  50   0   0    110    254   0         0     159       0       0.0
1024  54   1   0    120    188   0         1     113       0       1.4

   slope  ca  thal  target
1020     2   0     2       1
1021     1   1     3       0
1022     1   1     2       0
1023     2   0     2       1
1024     1   1     3       0
```

```
# Number of rows and columns in the dataset
print("\nShape of dataset:", heart_data.shape)
```

Shape of dataset: (1025, 14)

```
# Getting some info about the data
print("\nDataset Info:")
heart_data.info()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   age         1025 non-null  int64
 1   sex         1025 non-null  int64
```

```

2   cp      1025 non-null   int64
3   trestbps 1025 non-null   int64
4   chol     1025 non-null   int64
5   fbs      1025 non-null   int64
6   restecg  1025 non-null   int64
7   thalach  1025 non-null   int64
8   exang    1025 non-null   int64
9   oldpeak  1025 non-null   float64
10  slope    1025 non-null   int64
11  ca       1025 non-null   int64
12  thal     1025 non-null   int64
13  target   1025 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 112.2 KB

```

```

# Checking for missing values
print("\nMissing Values:")
print(heart_data.isnull().sum())

```

```

Missing Values:
age      0
sex      0
cp       0
trestbps 0
chol     0
fbs      0
restecg  0
thalach  0
exang    0
oldpeak  0
slope    0
ca       0
thal     0
target   0
dtype: int64

```

```

# Statistical measures about the data
print("\nStatistical Summary:")
print(heart_data.describe())

```

```

Statistical Summary:

```

	age	sex	cp	trestbps	chol
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	0.695610	0.942439	131.611707	246.000000
std	9.072290	0.460373	1.029641	17.516718	51.59251
min	29.000000	0.000000	0.000000	94.000000	126.00000
25%	48.000000	0.000000	0.000000	120.000000	211.00000
50%	56.000000	1.000000	1.000000	130.000000	240.00000
75%	61.000000	1.000000	2.000000	140.000000	275.00000
max	77.000000	1.000000	3.000000	200.000000	564.00000

	fbs	restecg	thalach	exang	oldpeak
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	0.149268	0.529756	149.114146	0.336585	1.071512
std	0.356527	0.527878	23.005724	0.472772	1.175053
min	0.000000	0.000000	71.000000	0.000000	0.000000
25%	0.000000	0.000000	132.000000	0.000000	0.000000
50%	0.000000	1.000000	152.000000	0.000000	0.800000
75%	0.000000	1.000000	166.000000	1.000000	1.800000
max	1.000000	2.000000	202.000000	1.000000	6.200000

	slope	ca	thal	target
count	1025.000000	1025.000000	1025.000000	1025.000000
mean	1.385366	0.754146	2.323902	0.513171
std	0.617755	1.030798	0.620660	0.500070
min	0.000000	0.000000	0.000000	0.000000
25%	1.000000	0.000000	2.000000	0.000000
50%	1.000000	0.000000	2.000000	1.000000
75%	2.000000	1.000000	3.000000	1.000000
max	2.000000	4.000000	3.000000	1.000000

```

# Checking the distribution of target variable
print("\nTarget Value Counts:")
print(heart_data['target'].value_counts())
print("\n1 ---> Defective Heart")
print("0 ---> Healthy Heart")

```

```

Target Value Counts:
target
1      526
0      499
Name: count, dtype: int64

```

```
1 ---> Defective Heart
0 ---> Healthy Heart
```

Splitting the Features and Target

```
X = heart_data.drop(columns='target')
Y = heart_data['target']
```

Feature Scaling (Important for SVM & LR)

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Splitting the Data into Training & Test

```
X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.2, stratify=Y, random_state=2)
```

```
print("\nShapes:")
print("X.shape:", X.shape)
print("X_train.shape:", X_train.shape)
print("X_test.shape:", X_test.shape)
```

```
Shapes:
X.shape: (1025, 13)
X_train.shape: (820, 13)
X_test.shape: (205, 13)
```

MODEL 1: Logistic Regression

```
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, Y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

```
lr_train_pred = lr_model.predict(X_train)
lr_test_pred = lr_model.predict(X_test)
```

```
lr_train_acc = accuracy_score(Y_train, lr_train_pred)
lr_test_acc = accuracy_score(Y_test, lr_test_pred)
```

```
print(f"Accuracy on Training data : {lr_train_acc:.4f}")
print(f"Accuracy on Test data      : {lr_test_acc:.4f}")
```

```
Accuracy on Training data : 0.8585
Accuracy on Test data      : 0.8049
```

MODEL 2: Random Forest (BETTER MODEL)

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=2)
rf_model.fit(X_train, Y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=2)
```

```
rf_train_pred = rf_model.predict(X_train)
rf_test_pred = rf_model.predict(X_test)
```

```
rf_train_acc = accuracy_score(Y_train, rf_train_pred)
rf_test_acc = accuracy_score(Y_test, rf_test_pred)
```

```
print(f"Accuracy on Training data : {rf_train_acc:.4f}")
print(f"Accuracy on Test data      : {rf_test_acc:.4f}")
```

```
Accuracy on Training data : 1.0000
Accuracy on Test data      : 1.0000
```

MODEL 3: Support Vector Machine (SVM)

```
svm_model = SVC(kernel='rbf', random_state=2)
svm_model.fit(X_train, Y_train)
```

SVC

SVC(random_state=2)

```
svm_train_pred = svm_model.predict(X_train)
svm_test_pred = svm_model.predict(X_test)
```

```
svm_train_acc = accuracy_score(Y_train, svm_train_pred)
svm_test_acc = accuracy_score(Y_test, svm_test_pred)
```

```
print(f"Accuracy on Training data : {svm_train_acc:.4f}")
print(f"Accuracy on Test data      : {svm_test_acc:.4f}")
```

```
Accuracy on Training data : 0.9463
Accuracy on Test data      : 0.9122
```

Model Compariso

```
print(f"{'Model':<25} {'Train Acc':>10} {'Test Acc':>10}")
print(f"{'Logistic Regression':<25} {lr_train_acc:>10.4f} {lr_test_acc:>10.4f}")
print(f"{'Random Forest':<25} {rf_train_acc:>10.4f} {rf_test_acc:>10.4f}")
print(f"{'SVM (RBF Kernel)':<25} {svm_train_acc:>10.4f} {svm_test_acc:>10.4f}")
```

Model	Train Acc	Test Acc
Logistic Regression	0.8585	0.8049
Random Forest	1.0000	1.0000
SVM (RBF Kernel)	0.9463	0.9122

```
# Best model
accuracies = {'Logistic Regression': lr_test_acc,
              'Random Forest': rf_test_acc,
              'SVM': svm_test_acc}
best_model_name = max(accuracies, key=accuracies.get)
print(f"\nBest Model: {best_model_name} with Test Accuracy: {accuracies[best_model_name]:.4f}")
```

```
Best Model: Random Forest with Test Accuracy: 1.0000
```

Detailed Evaluation on Best Model (RF)

```
print("DETAILED REPORT - Random Forest")
print(classification_report(Y_test, rf_test_pred,
                           target_names=['Healthy Heart', 'Defective Heart']))
```

```
DETAILED REPORT - Random Forest
precision    recall  f1-score   support

Healthy Heart      1.00      1.00      1.00       100
Defective Heart    1.00      1.00      1.00       105

   accuracy
macro avg      1.00      1.00      1.00       205
weighted avg    1.00      1.00      1.00       205
```

Building a Predictive System (Random Forest)

```
print("PREDICTIVE SYSTEM - Using Random Forest")

input_data = (58,1,0,114,318,0,2,140,0,4.4,0,3,1)

# Change the input data to a numpy array
input_data_as_numpy_array = np.asarray(input_data)

# Reshape and scale
input_data_reshaped = input_data_as_numpy_array.reshape(1, -1)
```

```
input_data_scaled = scaler.transform(input_data_reshaped)
```

```
# Predict using Random Forest
prediction = rf_model.predict(input_data_scaled)
print(f"\nInput Data: {input_data}")
print(f"Prediction: {prediction}")
```

```
if prediction[0] == 0:
    print("The Person does not have a Heart Disease")
else:
    print("The Person has Heart Disease")
```

PREDICTIVE SYSTEM - Using Random Forest

Input Data: (58, 0, 0, 100, 248, 0, 0, 122, 0, 1, 1, 0, 2)

Prediction: [1]

The Person has Heart Disease

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, warnings.warn(